

Fast string matching algorithm

Ala'a Al-Howaide, Wail Mardini*, Yaser Khamayseh, Muneer Bani Yasin

Department of Computer Science
Jordan University of science and Technology
Irbid, Jordan

* mardini@just.edu.jo

Abstract— String matching refers to find all or some of the occurrences of a text (usually called a pattern) in another text. Many fields depend on string matching algorithms to extract, retrieve, categorize ...etc documents as a backbone of the information system such as information retrieval, data mining, text categorization ...etc. We propose a fast string matching algorithm that reduces the effect of the pattern length on the search time. Our algorithm can beat the best know algorithm (Wu-Manber) in some cases. The running complexity of the algorithm is $O(n/m)$.

Index Terms—String matching, algorithms, searching, design of algorithms.

I. INTRODUCTION

String matching [1] consists in finding one/all the occurrences of a string (more generally called a pattern) in a text. The pattern P consists of m characters is denoted by $P=P[1 \dots m]$. Similarly, the text is denoted by $T=T[1 \dots n]$ with length of n characters. Both strings are built over a finite set of character called an alphabet.

Most of the string matching algorithms perform a preprocessing phase on the pattern to reduce the time needed later to match.

String-matching algorithms have the following general procedure. It starts by scanning the text using a window size equal to m . At the beginning it aligns the left ends of the window and the text, and then it compares the characters of the window with the characters of the pattern. After a whole match (*valid shift*) or a mismatch (*invalid shift*) of the pattern with the text, it shifts the window to the right. It repeats the previous procedure again until the left end of the window at the position $n-m$ of the text which is the last possible index for matching.

Different algorithms differ in the comparison step to find the valid on invalid shifts.

The rest of the paper is organized as follows. The next section presents related works on string matching algorithms. Section III introduces the idea and pseudo-code of our new algorithm. Performance comparison results are presented in section IV. Section V presents conclusion and future works.

II. RELATED WORK

The *naïve* algorithm [1], which is a single pattern string-matching algorithm, finds all valid shifts using a loop to check the condition $P[1..m]=T[s+1..s+m]$ for each $n-m+1$ possible shifts. The running time of this algorithm is $O(m(n-m+1))$.

To avoid the quadratic number of character comparisons performed by the naïve algorithm, *hashing* usually used to reduce the cost of comparison.

The idea is to check only if the contents of the window “look like” the pattern. So a hashing function is used to check the similarity between these two texts; the pattern $P=P[m]$ and the text $T[s+1..s+m]$. This idea was first introduced by *Rabin-Karp (RK)*. This algorithm computes the hash value of the next substring in constant time $O(1)$ which can be accomplished by using the advantage of a modulo operation. The hash value of the pattern is computed once as a preprocessing step. When equality between the hash value of the pattern and the hash value of the current substring is found, then it is necessary to perform an exact match check because hash value equality does not mean exact match. This has the cost of $\Theta(m)$ [4]. The running time of this algorithm is also $O(m(n-m+1))$ since at the worst case a hashing value would be equal and an exact match check should be performed.

The *Knuth Morris Pratt (KMP)* algorithm makes use of the information gained by previous symbol comparisons. It never re-compares a text symbol that has matched a pattern symbol. As a result, the complexity of the searching phase is $O(n)$. However, a preprocessing on the pattern is necessary in order to analyze its structure. The preprocessing phase has a complexity of $O(m)$. Since $m \leq n$, the overall complexity of the Knuth-Morris-Pratt algorithm is in $O(n)$ [1].

Wu and Manber (WM) [3] introduced an algorithm for multi pattern searching, which was enhanced to work with single pattern searching by *Lecroq* [2]. The idea was to hash the pattern by hashing $|P|/q$ of substrings each of size q in a cumulative manner using a hashing function which produces values within 0 and 255. Then it uses a function to determine the shift amount. This is the preprocessing phase required by this algorithm and it costs $O(m/q)$. In the searching phase if the shift value of a text's substring's hash value > 0 , the shift amount is applied else an exact match is performed. The running time of this algorithm is $O(n/q)$.

III. THE NEW ALGORITHM

The idea of the new algorithm is to search for the first pattern's character within the text. If it was found then it checks the character at distance m if it was also equal to the last pattern's character then it checks the rest of the characters within the substring naively. If a full match was found then it will shift the window m characters to the right. Considering that all the shifts values from $s+1$ to $s+m-1$ are invalid shifts. For example, if $P=aaab$, $T=aaabcdf$, and we found that when

the shift amount equal 1 is valid then none of the shifts 2, 3, or 4 are valid, since $T[4]=b$. But if we consider the example where $p="aaa"$ $T="aaaaaaa"$. The suggested algorithm will find two valid shifts values at 1 and 4 while there are 5 valid shifts values at 1, 2, 3, 4 and 5, this case happen only when the text is constructed of repeating the same pattern. Notice this is not the case in the natural English and it is not a very common case in other datasets such as DNA sequences and binary.

The algorithm is presented in Figure 1. It assumes that pattern's characters are mutually independent (doesn't account for characters dependences). By this the occurrence of any character will not effect or give an evidence of another character occurrence. So all characters within the pattern and the text have the same weight and appear to be random. Then selecting any pair of characters will results in average the same results and there will be $m-2$ characters to be checked if the two conditions has been satisfied.

Selecting the first and the last pattern's characters to give an evidence of a match allows the usage of a simple loop for checking the remaining $m-2$ characters.

This algorithm performs in worst case as *KMP* algorithm and it is very simple to implement where there is no preprocessing phase such as most of the algorithms.

```

Algorithm NEW( $T, P, m, n$ )
for  $s \leftarrow 0$  to  $n-m$ 
  if  $P[0]=T[s]$  and  $P[m-1]=T[s+m-1]$ 
     $j \leftarrow 1$ 
    while  $j < m-1$  do
      if  $P[j]=T[s+j]$ 
        if  $j=m-2$  then
          print "Pattern occurs with shift"  $s$ 
           $s+=m-1$ 
           $j++$ 
        else
           $j=m$ 
     $s++$ 

```

Figure 1. The new string matching algorithm.

The running time for the *for* loop is $O(n-m)$ when there is no matches at either or both of the two characters (first and last characters). And it will perform n/m steps when there is a match at each $s+m$ shift value, so the running time in this case will be $O(n/m)$, Notice that the algorithm doesn't go upon any character more than once.

It worth noting that the worst case would result in similar performance of the naïve algorithm however the case is not applicable in the typical string matching applications. An example for the worst case would occur when $P="aaaaaba"$ and $T="aaaaaaa..."$. Here, the algorithm will predict a match by matching the first and last characters, so it is going to check the rest of the characters naively. This will cause a running time $O(nm)$.

IV. EXPERIMENT RESULTS

To measure the efficiency of our string matching algorithm we perform several experiments with the previously discussed algorithms on different data sets.

A. Algorithms and environment

We conducted our experiments of the following four algorithms:

- The naïve algorithm.
- Rabin-Karp algorithm.
- KMP algorithm.
- The enhanced Wu-Manber algorithm for single string matching.

The last algorithm has been tested with another 17 different algorithms (BF, BM2Fast, TBM, SSABS, ZT, FS, BOM2, BNDM, FAOSO, SBNDM, SBNDM2, ... etc) in [2]. The results show that the enhanced Wu-Manber was the best in most of the cases. For example, in short pattern length experiments it outperforms all the algorithms over the binary, the E.coli, and the small alphabet size. For long pattern length experiments it outperforms all the algorithms over the binary alphabet (at lengths 32, 64, 128, and 256), the E.coli (at lengths 32-128), the small alphabet, and the English text.

In this paper, the four previously mentioned algorithms have been implemented and tested in C++ on different data sets (explained in the next subsection). The experiments have been ran on a Pentium(R) 4 CPU 3.20 GHz, 1MG cache and 2 GB of RAM running MS windows XP professional V.2 service pack 2. Number of runs was chosen to satisfy a 95% confidence level [5].

$$\text{Number of runs} = ((z * \pi * (1 - \pi))^{1/2})^2 * z$$

where z is a constant equal 1.96 and π is the correctness probability parameter which is usually used to 0.5.

The search's running time has been measured using the *clock* function in C++ [6].

B. Data sets

We performed our experiments on different types of text: text consists of a binary alphabet, text consists of an alphabet of size 12, text consists of a genome contents, and text consists of an English language alphabet:

- Binary alphabet and alphabet of size 12: the texts are constructed of 4,000,000 randomly built characters.
- Genome: is a DNA sequence composed of the four nucleotides bases *A*, *C*, *G*, and *T*, Adenine, Cytosine, Guanine, and Thymine respectively. We used the file *E.coli* of the *Large Canterbury Corpus* [7], which is constructed of 4,638,690 bases.
- Natural language: We used the file *world192.txt* of the *Large Canterbury Corpus*, which constructed of 2,473,400 of 94 different characters.

We take into account both the short patterns (sizes 5, 10, 20, and 30 characters) and long patterns (sizes 2^5 to 2^{10}) for each pattern length. We repeated the experiments until we reach the 95% of confidence interval as explained before. The results were then averaged and explained in the next subsection.

C. Results

To study the effect of the short and long pattern on the performance of the string matching algorithm, we have conducted two main sets of experiments. Figures 2 to 5 show the short pattern results and Figures 6 to 9 shows the long pattern results.

For short patterns, Figure 2 presents the results on the *E.coli* genome data set for different pattern sizes. The new algorithm outperforms the KMP for pattern sizes 20 and 30. The new algorithm overcome WM at pattern size of 5 and it overcomes the KMP at pattern size of 30 on the English data set, as shown in Figure 3. Figure 4 shows that the new algorithm performs nearly similar to KMP with a slightly overcome at pattern sizes 10, 20, and 30 on the binary alphabet. For alphabets of size 12, the new algorithm overcomes WM at pattern size 20 and 30, as shown in Figure 5. Notice that WM was not able to find all the occurrences of patterns of length 5 and 10 for most of the tests at $q=3$ (nearly the half of occurrences were missed).

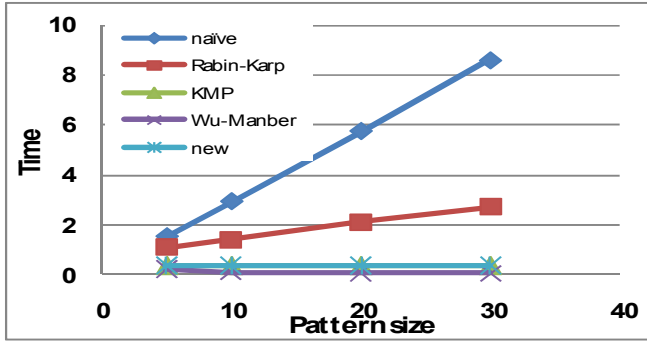


Figure 2. Results for short pattern on *E.coli* genome.

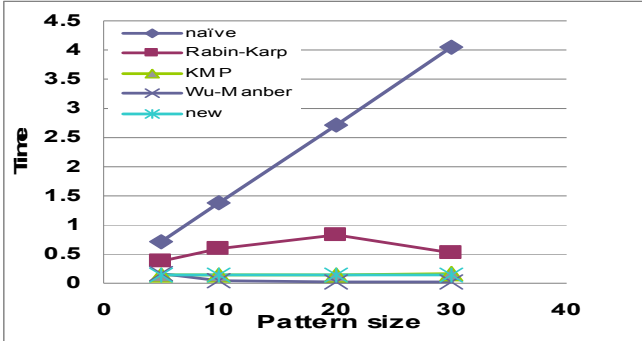


Figure 3. Results for short pattern on an English text.

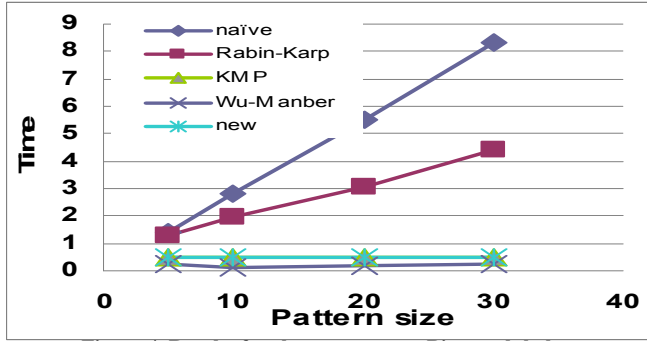


Figure 4. Results for short pattern on Binary alphabet.

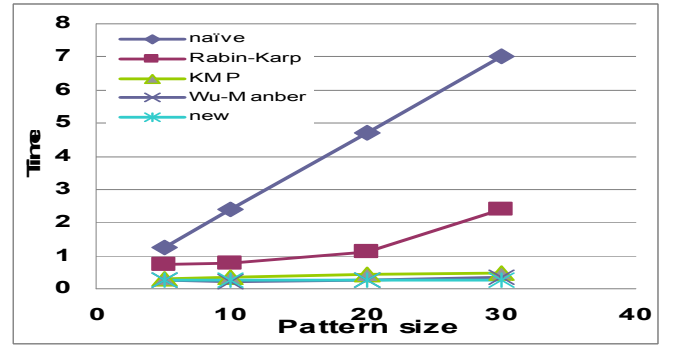


Figure 5. Results for short pattern on alphabet of size 12.

For long patterns, Figure 6 and Figure 7 show that the new algorithm outperform completely the KMP for all pattern size on *E.coli* genome data set and English data set, consecutively. The new algorithm was the fastest on the binary alphabet data set for pattern sizes from 64 to 1024 and on the alphabet of size 12 data set for all pattern lengths it was the fastest, as shown in Figure 8 and Figure 9 consecutively.

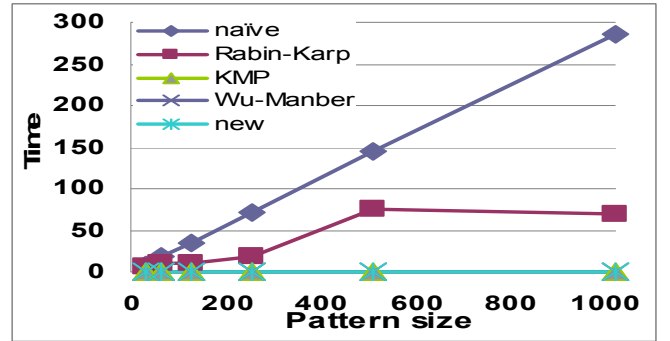


Figure 6. Results for long pattern on *E.coli* genome.

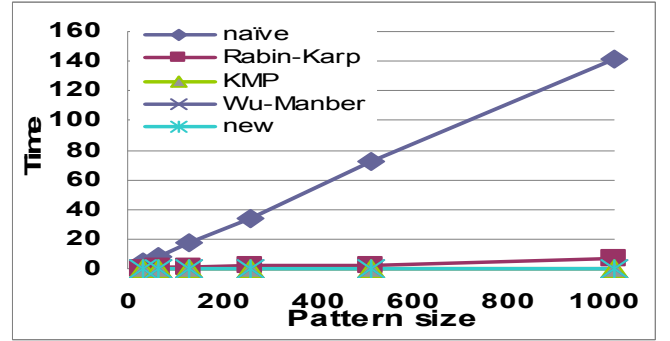


Figure 7. Results for long pattern on an English text.

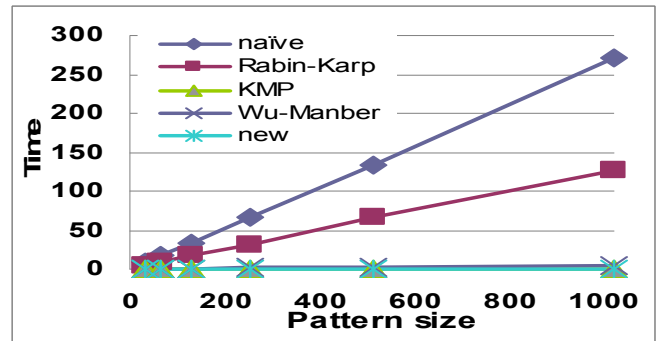


Figure 8. Results for long pattern on Binary alphabet.

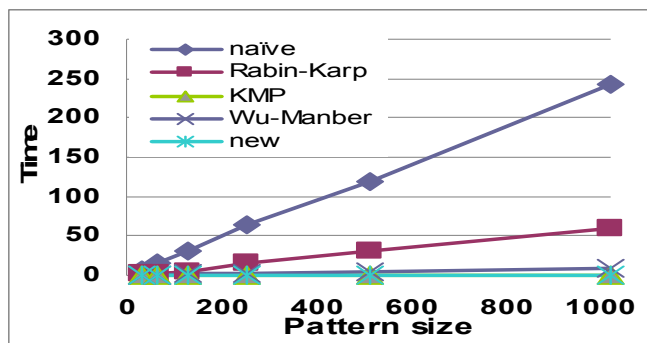


Figure 9. Results for long pattern on alphabet of size 12.

V. CONCLUSION AND FUTUR WORK

In this paper, we propose a new string matching algorithm for single pattern matching. In our algorithm, the first and last characters of the pattern were used as matching predictors for a pattern match within a text. It is simple where no needs for preprocessing phase or complicated calculations. Also the results show that it is efficient where it overcomes one of the fastest known algorithms in single string matching, WM, when

searching for long patterns as it doesn't uses either any extra memory space or massive CPU operations.

This algorithm has potential applications when resources such as memory and CPU are very limited on the running machine. Our future work includes improving the algorithm to handle the case where the text is structured of a repeated pattern. And also extending it by adapting the rule of distinguishing pattern's characters. by making it more dynamic which will give it an advantage over repeated pattern structure texts.

REFERENECES

- [1] T.Cormen, C.Leiserson, R.Rivest and C.Stein "Introduction To Algorithms", Second edition, 2007.
- [2] T.Lecroq "Fast Exact String Matching Algorithms", vol. 102, pp.229-235, 2007, Elsevier.
- [3] S.Wu and U.Manber "A Fast Algorithm For Multi-Pattern Searching", 1994, NSF.
- [4] C.Charras and T.Lecroq "Exact String Matching Algorithms". <http://www-igm.univ-mlv.fr/~lecroq/string/>
- [5] D.Wackerly, W.Mendenhall and R.Scheaffer "Mathematical Statistics With Applications", Fifth Edition, 1996.
- [6] H.M.Deitel and P.J.Deitle "C++ How To Program", Fourth Edition, 2003.
- [7]<http://www.data-compression.info/Corpora/CanterburyCorpus/>, 2010.